Crowd_Frame: A Simple and Complete Framework to Deploy Complex Crowdsourcing Tasks Off-the-shelf

Michael Soprano
University of Udine
Udine, Italy
michael.soprano@uniud.it

Kevin Roitero
University of Udine
Udine, Italy
kevin.roitero@uniud.it

Francesco Bombassei De Bona
University of Udine
Udine, Italy
bombasseidebona.francesco@spes.uniud.it

Stefano Mizzaro
University of Udine
Udine, Italy
mizzaro@uniud.it

ABSTRACT

Due to their relatively low cost and ability to scale, crowdsourcing based approaches are widely used to collect a large amount of human annotated data. To this aim, multiple crowdsourcing platforms exist, where requesters can upload tasks and workers can carry them out and obtain payment in return. Such platforms share a task design and deploy workflow that is often counter-intuitive and cumbersome. To address this issue, we propose Crowd_Frame, a simple and complete framework which allows to develop and deploy diverse types of complex crowdsourcing tasks in an easy and customizable way. We show the abilities of the proposed framework and we make it available to researchers and practitioners.

CCS CONCEPTS

• **Information systems** → **Crowdsourcing**; • **Software and its engineering** → **Software architectures**.

KEYWORDS

Crowdsourcing, Framework, User Behavior

ACM Reference Format:
Michael Soprano, Kevin Roitero, Francesco Bombassei De Bona, and Stefano Mizzaro. 2022. Crowd_Frame: A Simple and Complete Framework to Deploy Complex Crowdsourcing Tasks Off-the-shelf. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining (WSDM '22), February 21–25, 2022, Tempe, AZ, USA.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3488560.3502182

1 INTRODUCTION

Nowadays, crowdsourcing based approaches are commonly used to tackle individual or organizational tasks that require human intelligence [6]. To support the ever increasing need for crowd powered data gathered using outsourced tasks several platforms emerged, such as Amazon Mechanical Turk. These platforms aim to help requesters to outsource their tasks to a diverse, distributed, and large workforce able to perform each task. Indeed, although an increasing amount of published studies show the usage of Amazon Mechanical Turk [7], alternatives are also used [13]. Popular alternatives include Prolific, a platform dedicated to the scientific community where the crowd workers are explicitly recruited for participation in research tasks [11], or Toloka, a platform mainly focused on data labelling tasks.

A task requester is an individual who wants to deploy a crowdsourcing task on a chosen platform. The workflow involves several phases. Initially, the requester sets some parameters such as the number of crowd workers required, the time allowed for each worker to perform the task, and so on. Then, s/he designs the task layout. Usually, a markup language is provided to help to build the user interface. The requester must write the logic to handle the task using client side programming. Once the project is finalized the task can be deployed an arbitrary amount of times using a support file to vary the input data. Each instance of a task assigned to a worker is usually called HIT (Human Intelligence Task). A set of HITs is called batch [12]. When a worker completes the HITs assigned, the requester can approve and thus pay the worker or reject the final submission. This workflow shows several difficulties across all platforms: the requester must have advanced programming skills; the user interface is built by mixing the presentation and business logic; the input data passing mechanism is often cumbersome, and the responsibility to store the data produced by each worker lies on the requester for non trivial experiments. A solution to these problems is to rely on crowdsourcing platforms only to recruit the workforce. The recruited workers reach the task deployed using external software in a private platform to perform it. Then, they return to the crowdsourcing platform to receive the payment. The task design and deployment processes can be handled by external software. We developed Crowd_Frame, which is a software system that allows us to easily design and deploy diverse types of crowdsourcing tasks. We make it available to the research community. It is freely downloadable at: https://github.com/Miccighel/Crowd_Frame.

2 CASE STUDY

We present in the following a case study that we use to describe the capabilities of Crowd_Frame. Roitero et al. [16] use Crowd_Frame to deploy a misinformation assessment task. Their goal is to understand if the crowd workers can identify and correctly classify online misinformation concerning the COVID-19 pandemic. Their task uses two questionnaires to collect workers' backgrounds and to estimate workers' cognitive abilities. Then, each worker is shown with a set of statements. A six-level assessment scale is used to provide truthfulness labels. Each worker must provide a URL using a custom search engine and a textual justification to support the rationale behind each assessment. Figure 1 shows the resulting interface. A requester can easily replicate such a setup using

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WSDM '22, February 21–25, 2022, Tempe, AZ, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9132-0/22/02…$15.00
https://doi.org/10.1145/3488560.3502182

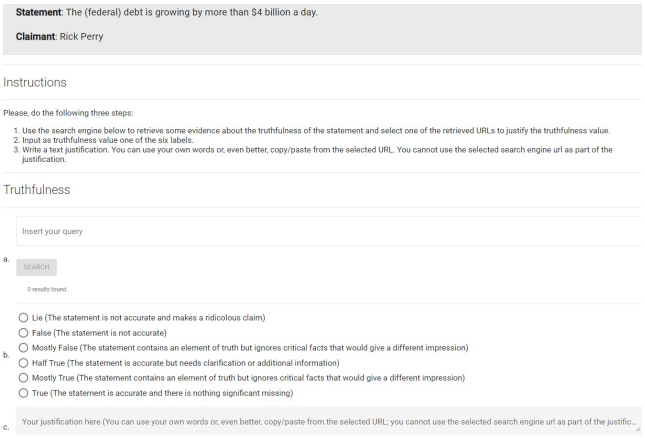

Figure 1: Worker interface of a misinformation assessment task deployed using Crowd_Frame.

Crowd_Frame. The required steps are detailed in Section 3. Other tasks have been deployed using Crowd_Frame [14, 15, 18].

3 SYSTEM DESIGN

Crowd_Frame copes with the difficulties of the crowdsourcing activity workflow. Requesters can instantiate and configure each task using a simple user interface. When the requester completes the configuration, s/he deploys the task into a controllable and customizable environment. Then, the requester uses a wrapper as a bridge between the crowd workers recruited and the task deployed. The crowd workers can complete their tasks and return to the crowdsourcing platform to receive the payment.

Architecture. Crowd_Frame is a client-side application developed using Angular, an open source framework for web development. Figure 2 shows its architecture. Crowd_Frame is composed of four main components namely Generator, Skeleton, Search Engine, and Logger. Crowd_Frame relies on Amazon S3 to store tasks configuration, data produced by crowd workers, and each deployment. We choose to use such a storage provider, but other choices are possible and easy to implement, including deployment on private servers. A requester uses the Generator to configure the task. The configuration is uploaded to a private S3 bucket (i.e., a storage resource). Then, the requester can publish a set of HITs on the chosen crowdsourcing platform. Each crowd worker interacts with the wrapper, which redirects the worker to the application deployed on a public S3 bucket. The worker interacts with the Skeleton to perform the task. The Skeleton may embed a custom and configurable Search Engine. The Logger analyses the worker's behavior during the task. The data produced during the task execution is stored on buckets.

Generator. The Generator component allows customizing the configuration of a crowdsourcing task, as shown in Figure 2. The user interface is a form composed of eight separate steps. Within each step, a requester configures a certain aspect of the task. The first step concerns questionnaires. It allows creating one or more questionnaires that workers will fill before or after task execution, depending on the requester's needs. Three different types of questionnaires are available and it is easy to implement additional types.

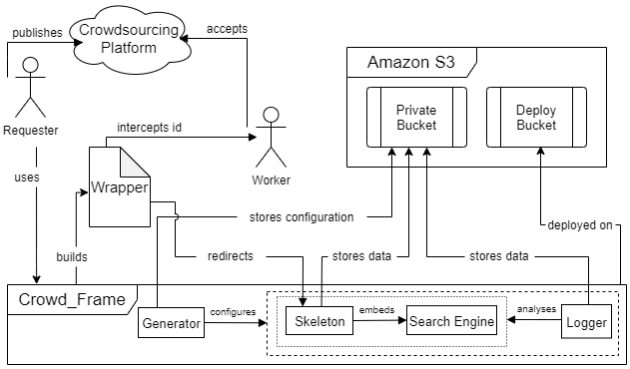

Figure 2: Architecture of a crowdsourcing task deployed using Crowd_Frame.

The standard option allows implementing simple questionnaires where the worker responds to questions with a predefined and limited number of alternative answers. The CRT option allows to implement Cognitive Reflection Tests [3], often used to estimate workers' cognitive abilities. The Likert option allows implementing questionnaires where workers choose from a range of possible responses to a specific question. It is possible to easily extend the Generator to add additional types of questionnaires. The task outlined in Section 2 uses a single standard questionnaire and a single CRT questionnaire. The second step concerns the *evaluation dimensions*. It allows configuring what the worker will assess for each element of the HIT assigned. The task outlined in Section 2 requires workers to assess misinformation of political statements. Thus, a dimension that workers must evaluate is the truthfulness of each statement. A dimension requires an evaluation scale. Every possible type of evaluation scale is available in Crowd_Frame. A requester can configure categorical, interval, ratio scales such as magnitude estimation. Each dimension may also require the worker to provide a textual justification or a URL to support the assessment. Furthermore, each dimension may be required to satisfy a quality check at the end of the task. The task outlined in Section 2 requires evaluating a statement that is obviously true with a higher truthfulness value than a statement that is obviously false.

The third step concerns task instructions. The requester writes the set of instructions using a rich textual editor. The worker is shown with such instructions before the task. Similarly, the fourth step concerns evaluation instructions. The worker is shown with such instructions within the task's body. Such instructions explain how the worker should assess each element of the hit assigned along each dimension. The evaluation instructions of the task outlined in Section 2 explain to each worker that they must select a truthfulness value, provide a URL and write a textual justification. The fifth step concerns the Search Engine. It allows choosing the search provider wanted and adding a list of domains to filter from search results. The task outlined in Section 2 filters out search results originated from two fact-checking websites. The sixth step concerns general task settings. The requester can set the maximum amount of tries for each worker. This allows workers to revise their work if they do not pass the quality checks at the end of the task

and helps in reducing task abandonment [4, 5]. The requester can also configure the minimum time that each worker must spend on each HIT element. Such a check is useful to prevent automated bots from completing a task. The requester can set a countdown to limit the time available for the worker to complete the assessment. Maddalena et al. [10] show that such a constraint leads to an improvement in worker quality. It also allows optimizing the cost of a crowdsourcing task. The requester can enable an annotation interface to require the worker to annotate and label texts, such as for social media conversation annotation [19]. The requester may choose previous batches of workers to block for the current task. For example, this can be useful when performing a longitudinal study, where the same task is repeated multiple times with new workers only [15]. Lastly, the requester is required to upload a file containing the HITs for the current task. The file must comply with a special JSON format and must be generated independently. The seventh step concerns additional checks on workers. The requester can manually specify a set of worker identifiers to allow or block for the current task. It allows to fine tune the admitted workers without blocking an entire batch. The eighth and last step shows a summary of the configuration created. The requester can upload the configuration to the private S3 bucket if satisfied.

Wrapper. Figure 2 shows the presence of a wrapper between the workers recruited on a crowdsourcing platform and a task deployed using Crowd_Frame. The mechanism is used to identify a worker and assign a HIT to him/her relies on the possibility to provide input variables to the platform. For instance, Amazon Mechanical Turk allows such by uploading a special CSV file. The columns of the file are the variables and each row provides its values. The mechanism for other platforms is similar. The requester uploads a file input and output tokens. The input token is an alphanumeric string used to match the worker with a certain HIT. The output token is used by the crowd worker to submit his/her work. The wrapper shows the input token to the worker and fetches his/her platform specific identifier. When the worker reaches the deployment for the current task s/he must paste the input token. If it is correct, the worker can perform the task. Then, s/he is shown with an output token when successfully completing the task. Such a token must be copied back on the wrapper. If the token matches the one provided, the worker can be paid. Worker identifier and input token are fundamental to ensure correct matching between workers and HITs.

Skeleton. The Skeleton is the core component of Crowd_Frame. Figure 3 details the interaction between a crowd worker and the Skeleton itself. The worker reaches the application deployed on the public bucket through the wrapper. His/her identifier is sent as a URL parameter. The worker knows the input token. When the worker reaches the current deploy the identifier is inserted into a list. This allows preventing a worker from performing the same task a second time. If the worker identifier is already present, s/he is prevented from performing the task. The requester can disable this access control mechanism if needed. Then, the worker pastes the input token. If the token is valid, s/he is shown with the general instructions and the task is unlocked. The Skeleton fetches the task configuration from the private bucket and instantiates the layout required. Then, the worker performs the task configured by the requester. The Skeleton creates a page for each questionnaire and

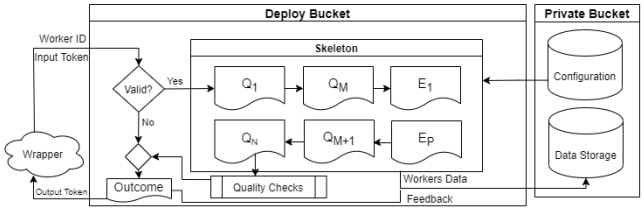

Figure 3: Worker execution flow of a task deployed using Crowd_Frame.

element. The appearance of each page depends on the dimensions configured. The evaluation instructions are shown on every page. The worker can go back and forth on each element and perform the work required. When the worker is finished, the quality checks trigger. If the worker satisfies each check, s/he is shown with the output token. The worker may also write textual feedback for the requester. Finally, the worker can paste the output token back to the wrapper. During the whole execution flow, the Skeleton stores the data produced by the worker on the private bucket.

Search Engine. Crowd_Frame implements a component that allows integrating a customizable search engine within the task body. The requester can require the worker to provide an URL for each evaluation dimension during task execution. The component implements Bing Web Search[1] and PubMed Search[2] APIs. The requester chooses the most suitable search provider and can provide a list of domains to filter from search results during the task's configuration. The worker may look for evidence using the Search Engine without leaving the task's interface. Additional search providers can be easily implemented.

Logger. Crowd_Frame implements a logging component that allows capturing worker behavior during the task. Such component captures user actions generated by the worker such as mouse movements, keystrokes, etc. The requester can choose which user actions to monitor. The Logger uses Directives, which are classes that can dynamically add additional behavior to a custom markup element. The log messages produced when capturing user events are stored using a cloud based logging server which relies on the infrastructure provided by Amazon Web Services. Each user action captured is sent via HTTP to an API gateway. The gateway redirects the message's body to a queue. A server-less function polls the queue in search of new messages and collects batches of up to 100 messages. The function parses each log message and the user action is stored as a record into a database table. The whole logging system back-end can be deployed also on a private server. The requester can set a custom endpoint to where each log message will be sent. The performances of the AWS based pipeline have been tested using a cluster of 100 Amazon Elastic Compute Cloud instances. The chosen type of instance was a t3.micro, which features two cores from an Intel Xeon Platinum 8000 series processor with a clock speed of up to 3.1 GHz, 1 GB of RAM, and up to 10 Gb/s of network burst bandwidth. To test server load, each instance of the cluster spawned two processes to send log messages. Initially, 200 processes were spawned. Each process sent a message every

[1] https://www.microsoft.com/en-us/bing/apis/bing-web-search-api
[2] https://www.ncbi.nlm.nih.gov/pmc/tools/developers/

10 ms and the test lasted for five minutes. The test lead to 16538 Lambda invocations to manage the whole set of log messages. The estimated mean number of requests processed each second by the pipeline is 5512. A negligible amount of messages were lost, due to network communication. The pipeline can scale in size. Furthermore, Lambda can en-queue again a message if processing errors arise. The pipeline has been tested also on a dedicated server using an EC2 instance based on a 2nd generation AMD EPYC processor with 4 cores/8 threads, running at frequencies up to 3.3 GHz, and having 16 GB of RAM. The server managed to process, without losing messages, up to 5000 requests/s. Over this limit, the server goes in overloading failing progressively to accept new requests. The mean (median) time estimated between each log message to reach such a limit is 2.76 (1) seconds. Thus, at least 5000 workers working at the same time are required to overload the server.

Deployment. A requester needs an AWS account to deploy crowdsourcing tasks. S/he must fill an environment file to provide the name of each bucket, admin user credentials and more, and then uses an ad-hoc script to build and deploy the application. The GitHub repository contains detailed instructions.

4 RELATED WORK

Several tools that aid requesters during the whole crowdsourcing activity exist. CloudResearch [9] is a research platform that integrates with Amazon Mechanical Turk that aims to improve the quality of the crowdsourcing data collection process. CrowdForge [8] is a general purpose framework for accomplishing complex human computation tasks. It is based on the idea that complex work can be broken up into small and independent pieces while the system manages their coordination dependencies. iCrowd [2] is an adaptive crowdsourcing framework that estimates in real time the accuracy of workers by evaluating their performances on completed tasks. It can be used before the task's launch to choose the best workers available. CrowdTruth 2.0 [1] is a method to aggregate crowdsourcing responses after the task using disagreement aware metrics. It can be used to leverage workers' data processing after the task. CrowdEIM [17] is a tool based on mobile social media platforms which allow crowdsourcing information during emergencies.

Crowd_Frame supports diverse crowdsourcing platforms and allows to recruit general purpose crowd workers. Its configuration mechanism allows to easily decompose a complex task into several intermediate steps without the need for coordination mechanisms. All the data produced during task performing are securely stored. The Logger component allows studying user behavior. Furthermore, Crowd_Frame allows to easily implement various types of tasks, such as misinformation assessment, emergency information crowdsourcing, span annotations, and more.

5 CONCLUSIONS

We described the Crowd_Frame software that aims to help requesters to design and deploy diverse types of crowdsourcing tasks. Crowd_Frame has two main limitations. First, it does not have a mechanism to allocate a dataset into an arbitrary amount of HITs. This means that the requester must build manually the required HITs. If each element must be evaluated by more than one worker, the allocation scheme can be complex. In the future, we will develop a tool that automatically allocates a dataset depending on the number of workers required for each element. Second, Crowd_Frame does not have an interface to monitor the progress of a task deployed: a requester must use the user interface of the crowdsourcing platform chosen to understand how many more HITs have to be evaluated. In the future, we will develop a user interface to monitor a task's progress. It will interact with the crowdsourcing platform chosen and will provide feedback within Crowd_Frame. We will also implement the possibility to use other providers to store task configurations and data such as Microsoft Azure. We also aim to allow requesters to use private servers to deploy Crowd_Frame.

REFERENCES

[1] Anca Dumitrache, Oana Inel, Lora Aroyo, Benjamin Timmermans, and Chris Welty. 2018. CrowdTruth 2.0: Quality Metrics for Crowdsourcing with Disagreement. In *Proceedings of the 1st SAD Workshop, and Proceedings of the 1st CrowdBias Workshop co-located the AAAI HCOMP Conference*, Vol. 2276. 11–18.

[2] Ju Fan, Guoliang Li, Beng Chin Ooi, Kian-lee Tan, and Jianhua Feng. 2015. iCrowd: An Adaptive Crowdsourcing Framework. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1015–1030.

[3] Shane Frederick. 2005. Cognitive Reflection and Decision Making. *Journal of Economic Perspectives* 19, 4 (12 2005), 25–42.

[4] Lei Han, Kevin Roitero, Ujwal Gadiraju, Cristina Sarasua, Alessandro Checco, Eddy Maddalena, and Gianluca Demartini. 2019. All Those Wasted Hours: On Task Abandonment in Crowdsourcing. In *Proceedings of the 12th ACM International WSDM Conference*. 321–329.

[5] Lei Han, Kevin Roitero, Ujwal Gadiraju, Cristina Sarasua, Alessandro Checco, Eddy Maddalena, and Gianluca Demartini. 2021. The Impact of Task Abandonment in Crowdsourcing. *IEEE Trans. Knowl. Data Eng.* 33, 5 (2021), 2266–2279.

[6] Jeff Howe. 2006. *The Rise of Crowdsourcing*. Wired.

[7] Melissa G. Keith, Louis Tay, and Peter D. Harms. 2017. Systems Perspective of Amazon Mechanical Turk for Organizational Research: Review and Recommendations. *Frontiers in Psychology* 8 (2017), 1359.

[8] Aniket Kittur, Boris Smus, Susheel Khamkar, and Robert E Kraut. 2011. Crowdforge: Crowdsourcing complex work. In *Proceedings of the 24th annual ACM UIST Symposium*. 43–52.

[9] Leib Litman, Jonathan Robinson, and Tzvi Abberbock. 2017. TurkPrime.com: A versatile crowdsourcing data acquisition platform for the behavioral sciences. *Behavior Research Methods* 49, 2 (01 4 2017), 433–442.

[10] Eddy Maddalena, Marco Basaldella, Dario De Nart, Dante Degl'Innocenti, Stefano Mizzaro, and Gianluca Demartini. 2016. Crowdsourcing Relevance Assessments: The Unexpected Benefits of Limiting the Time to Judge. In *Proceedings of the 4th AAAI HCOMP Conference*. 129–138.

[11] Stefan Palan and Christian Schitter. 2018. Prolific.ac—A subject pool for online experiments. *Journal of Behavioral and Experimental Finance* 17 (2018), 22–27.

[12] Gabriele Paolacci, Jesse Chandler, and Panagiotis G Ipeirotis. 2010. Running experiments on Amazon Mechanical Turk. *Judgment and Decision Making* 5, 5 (2010), 411–419.

[13] Eyal Peer, Laura Brandimarte, Sonam Samat, and Alessandro Acquisti. 2017. Beyond the Turk: Alternative platforms for crowdsourcing behavioral research. *Journal of Experimental Social Psychology* 70 (2017), 153–163.

[14] Kevin Roitero, Michael Soprano, Shaoyang Fan, Damiano Spina, Stefano Mizzaro, and Gianluca Demartini. 2020. Can The Crowd Identify Misinformation Objectively? The Effects of Judgment Scale and Assessor's Background. In *Proceedings of the 43st International ACM SIGIR Conference*. 439–448.

[15] Kevin Roitero, Michael Soprano, Beatrice Portelli, Massimiliano De Luise, Damiano Spina, Vincenzo Della Mea, Giuseppe Serra, Stefano Mizzaro, and Gianluca Demartini. 2021. Can the crowd judge truthfulness? A longitudinal study on recent misinformation about COVID-19. *Personal & Ubiquitous Computing* (2021).

[16] Kevin Roitero, Michael Soprano, Beatrice Portelli, Damiano Spina, Vincenzo Della Mea, Giuseppe Serra, Stefano Mizzaro, and Gianluca Demartini. 2020. The COVID-19 Infodemic: Can the Crowd Judge Recent Misinformation Objectively?. In *Proceedings of the 29th ACM International CIKM Conference*. 1305–1314.

[17] Hongzhou Shen, Junpeng Shi, and Yihan Zhang. 2021. CrowdEIM: Crowdsourcing Emergency Information Management Tasks to Mobile Social Media Users. *International Journal of Disaster Risk Reduction* 54 (2021), 102024.

[18] Michael Soprano, Kevin Roitero, David La Barbera, Davide Ceolin, Damiano Spina, Stefano Mizzaro, and Gianluca Demartini. 2021. The Many Dimensions of Truthfulness: Crowdsourcing Misinformation Assessments on a Multidimensional Scale. *Information Processing & Management* 58, 6 (2021), 102710.

[19] Arkaitz Zubiaga, Maria Liakata, Rob Procter, Kalina Bontcheva, and Peter Tolmie. 2015. Crowdsourcing the Annotation of Rumourous Conversations in Social Media. In *Proceedings of the 24th International WWW Conference*. 347–353.